

AI ENGLISH SPEAKING COACH

Complete SaaS Product + Technical Build Specification

Personal-use first • Voice-first • Communication improvement • Agent-ready

Purpose: Build a focused AI English-speaking coach inspired by the useful learning patterns of Supernova and MySivi, but designed specifically to improve the user's real-world communication. This is NOT an AI chat app.

Design principle: Every feature must make the user speak, listen, respond, receive useful feedback, repeat, and improve.

1. Product Vision

Build a voice-first personal English communication coach that behaves like a structured speaking trainer rather than a general chatbot. The product should diagnose the learner, create a daily practice plan, conduct guided and free conversations, analyze speech, remember recurring weaknesses, and progressively train fluency, grammar, pronunciation, vocabulary, listening, confidence, and real-world communication.

The current Supernova product publicly describes AI-enabled spoken-English learning covering reading, listening, grammar and vocabulary with interactive activities. MySivi publicly emphasizes real-world conversations, personalized practice, pronunciation/grammar/vocabulary/fluency feedback, mistake memory and communication scenarios. This specification combines those useful patterns into one focused personal coach.

2. Product Scope — What It Is / Is Not

IN SCOPE	OUT OF SCOPE FOR V1
AI English speaking coach	General AI chat assistant
Voice-first lessons and conversations	Open-ended text chatbot
Speaking assessment and feedback	Unrelated productivity tools
Grammar, pronunciation, fluency, vocabulary	Data analytics / BI
Listening and dictation	Social network
Tamil ↔ English support	Large peer marketplace
Real-life roleplay and interviews	Complex billing/subscription system
Personalized daily plan	Enterprise administration
Mistake memory + spaced repetition	100+ third-party integrations
Progress and communication analytics	Avatar/video generation initially

3. Primary Goal

The product's single measurable outcome:

“I can speak English more naturally, clearly and confidently in real conversations.”

- Reduce hesitation and long pauses.
- Reduce mental translation from Tamil/native language into English.
- Improve sentence formation and grammar while speaking.
- Improve pronunciation, stress, rhythm and clarity.
- Increase usable vocabulary through context.
- Build confidence for interviews, meetings, presentations and daily conversations.
- Create a consistent speaking habit with short daily sessions.
- Show measurable improvement over weeks and months.

4. End-to-End User Workflow

FIRST USE



Create account / local profile

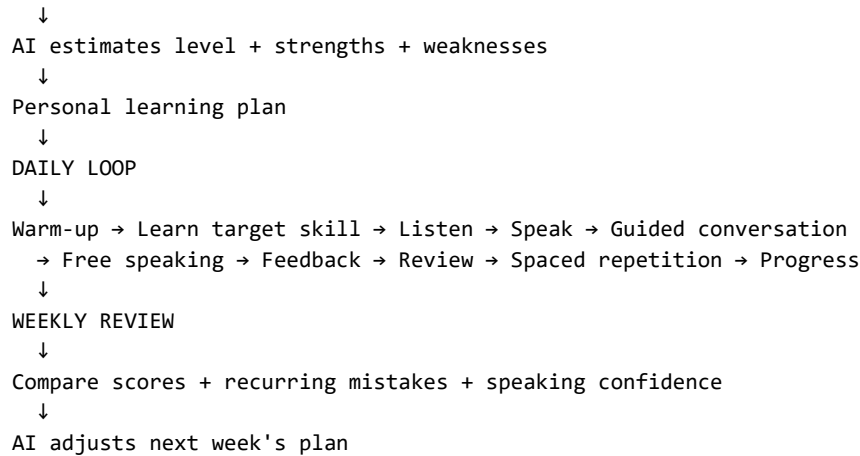


Select native language: Tamil (default) / other

Select learning goal: Communication / Interview / Workplace / General



Initial speaking assessment



5. Complete Feature Set

5.1 Onboarding & Initial Assessment

- Name/profile, native language, English level (optional self-selection), goals and preferred daily practice duration.
- Voice-based assessment using several prompts: self-introduction, daily routine, past event, opinion, future plan.
- Evaluate grammar, vocabulary, pronunciation/clarity, fluency, pauses, fillers, response length, listening comprehension and confidence proxies.
- Produce baseline report and initial personalized learning plan.

5.2 AI English Coach — Voice Only

- Voice-first interaction. User speaks; coach listens and responds with voice.
- Push-to-talk and continuous conversation modes.
- Interruptible AI response where technically supported.
- Configurable coach personality: supportive, strict, interview coach, workplace coach.
- Difficulty automatically adapts to learner level.
- Text transcript can be shown as an optional support layer, but the product is not a text-chat experience.

5.3 Guided Conversation

- Coach asks one question at a time.
- Conversation follows a target lesson objective, e.g. past tense, introductions, opinions or workplace phrases.
- AI should not constantly interrupt the learner. Capture mistakes and provide feedback at the right moment.
- Use follow-up questions based on the learner's answer.
- At the end, summarize 3–5 highest-value improvements.

5.4 Free Speaking / Extempore

- Prompt user to speak for 30 seconds, 1 minute, 2 minutes or 3 minutes.
- Topics: self, college, work, technology, family, travel, business, hobbies, opinions, current events, storytelling.
- Analyze fluency, structure, fillers, pauses, grammar patterns, vocabulary variety and clarity.
- Show what was good, what to improve, and one concrete exercise for the next session.

5.5 Real-Life Scenario Roleplay

- Job interview, HR round, technical interview, client meeting, sales call, presentation, networking, restaurant, hotel, airport, shopping, phone call, customer support, travel and workplace conversations.

- AI adopts a role and reacts naturally instead of following a rigid script.
- Scenario difficulty and vocabulary adapt to the learner.
- End with a communication score and targeted feedback.

5.6 Grammar Coaching

- Grammar should be learned through speaking, not textbook-only lessons.
- Detect tense, articles, prepositions, subject-verb agreement, word order, auxiliary verbs and common sentence errors.
- Show: learner sentence → corrected sentence → short reason → repeat task.
- Store recurring grammar mistakes for future practice.

5.7 Pronunciation & Speech Clarity

- Word and sentence pronunciation practice.
- Speech-to-text comparison is not enough for true pronunciation scoring; use a pronunciation-capable speech assessment service/model where available.
- Evaluate phoneme/word accuracy, stress, rhythm, pacing and intelligibility when supported.
- Give a short actionable correction rather than overwhelming the learner.
- Maintain a list of recurring difficult words/sounds.

5.8 Listening Practice

- AI speaks a short passage or dialogue.
- Ask comprehension questions.
- Include multiple-choice, short-answer and retell exercises.
- Dictation mode: hear sentence → type what was heard → compare.
- Difficulty increases based on performance.

5.9 Tamil → English Speaking Practice

- Generate natural Tamil prompts based on the lesson objective.
- User translates aloud into English.
- Evaluate meaning, grammar, naturalness and pronunciation.
- Show a natural English version and ask the user to repeat it.
- Progressively reduce translation support to encourage thinking in English.

5.10 Vocabulary Builder

- Capture useful words and phrases from conversations and mistakes.
- Teach vocabulary in context, not isolated word lists.
- For each item store meaning, example, learner sentence, pronunciation and review schedule.
- Use new vocabulary in future speaking scenarios.

5.11 Mistake Memory

- Every important mistake becomes a structured learning record.
- Store original sentence, corrected sentence, category, explanation, frequency, last seen, mastery and next review.
- Prioritize repeated mistakes over one-off mistakes.
- Never overload the user with every tiny error.

5.12 Spaced Repetition

- Review skills/phrases on a schedule such as Day 1, 2, 4, 7, 14 and 30, adjustable by performance.
- Failure shortens the interval; repeated success lengthens it.
- Review should be active speaking/listening, not just flashcards.

5.13 Daily Personalized Plan

- Default 15–30 minute plan.
- Example: 3 min warm-up → 5 min vocabulary/phrases → 5 min guided speaking → 5 min scenario → 5 min free speaking → 2 min feedback.
- Plan is generated from recent performance and long-term weaknesses.
- User can choose 10, 20, 30 or 45 minute sessions.

5.14 Progress Dashboard

- Overall communication score.
- Fluency, grammar, vocabulary, pronunciation/clarity, listening and confidence indicators.
- Speaking minutes, sessions completed, streak, words/phrases mastered and recurring mistakes.
- Weekly trend rather than obsessing over daily score fluctuations.
- Before/after comparisons and monthly progress report.

5.15 Interview & Workplace Coach

- Mock HR interview, self-introduction, project explanation, salary discussion, client meeting, presentation, team discussion and networking.
- Score clarity, structure, conciseness, confidence, vocabulary and grammar.
- Allow repeated attempts and compare attempt 1 vs attempt 2.

5.16 Reading-Aloud Mode

- Provide short passages appropriate to level.
- User reads aloud.
- Assess pronunciation/clarity where supported.
- Extract useful phrases and vocabulary.
- Optional shadowing: listen to model → repeat with similar rhythm.

5.17 Gamification — Lightweight

- Daily streak, session goal, XP, milestones and badges.
- Do not let gamification distract from speaking quality.

6. Learning Methodology

LISTEN → UNDERSTAND → SPEAK → RECEIVE FEEDBACK → REPEAT → REUSE

For every target skill:

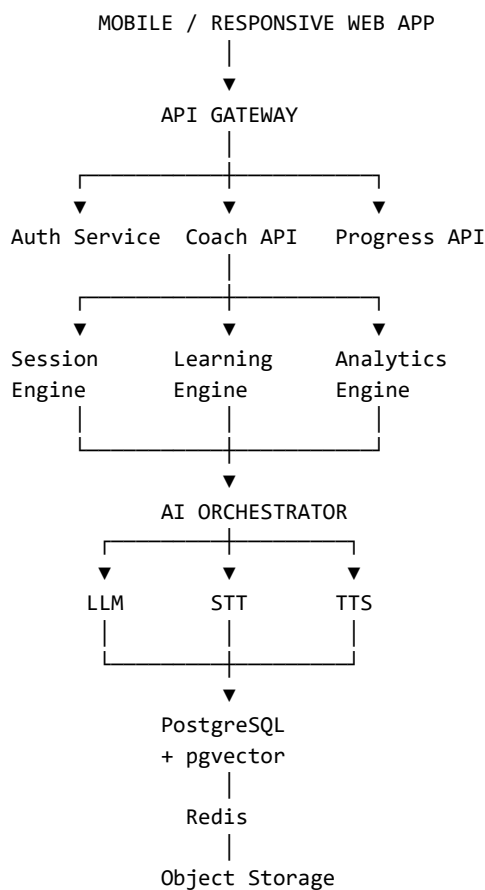
1. Introduce a small concept or phrase.
2. Give one or two examples.
3. Ask the learner to produce it.
4. Correct only the most important errors.
5. Make the learner say it again.
6. Reuse it in a different real-life context.
7. Schedule it for later review.

The coach should prioritize active speaking and repeated use over passive lessons.

7. AI Coach Behavior Specification

- Act like a patient professional English communication coach, not a chatbot.
- Prefer speaking over typing.
- Ask one question at a time.
- Keep responses short enough for voice interaction.
- Do not interrupt every mistake during natural conversation.
- During guided drills, corrections can be immediate.
- During free speaking, collect errors and give a concise end-of-session review.
- Correct for natural communication, not only textbook grammar.
- Explain difficult concepts in Tamil when the learner requests help or is clearly blocked.
- Encourage the learner to try again instead of immediately giving the answer.
- Use the learner's history to choose what to practice next.
- Avoid fake precision. If the speech system cannot reliably assess accent or phonemes, say so and use a safer clarity metric.

8. Technical Architecture



9. Recommended Tech Stack

Layer	Recommended	Purpose
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui	Responsive web app / PWA
Mobile later	Flutter or React Native	Native Android/iOS packaging if needed
Backend	Python + FastAPI	AI and application APIs
Database	PostgreSQL	Users, sessions, learning records, mistakes, progress
Vector search	pgvector	Optional semantic memory / content retrieval
Cache/queue	Redis	Sessions, jobs, rate limiting, background processing
Background jobs	Celery/RQ or FastAPI worker + Redis	Audio processing, reports, scheduled reviews
File storage	Cloudflare R2 / S3-compatible storage	Audio, reports and learning assets
LLM	Provider abstraction: Google Gemini API first; keep provider interfaces for future alternatives	Conversation planning, correction, lesson generation
STT	Speech-to-text API/model	Transcribe learner speech
TTS	Natural neural TTS API/model	Coach voice
Pronunciation	Speech/pronunciation assessment API/model	Pronunciation/clarity analysis
Charts	Recharts/ECharts	Progress visualization
Auth	Google OAuth + email/OTP optional	Authentication
Deployment	Vercel frontend + Render/Railway/Fly.io backend initially	Simple personal deployment
Monitoring	Sentry + structured logs	Errors and reliability

10. Core Database Schema

Table	Important fields
users	id, name, native_language, target_language, level, goals, daily_minutes
learning_profiles	user_id, baseline_scores, strengths, weaknesses, preferences
sessions	id, user_id, type, started_at, ended_at, duration, overall_score
turns	session_id, speaker, transcript, audio_url, timestamp
feedback	session_id, category, severity, original_text, corrected_text, explanation
mistakes	user_id, category, pattern, original, correction, frequency, mastery, next_review
vocabulary	user_id, word/phrase, meaning, example, pronunciation, mastery, next_review
skills	id, category, level, description, prerequisites
skill_progress	user_id, skill_id, score, attempts, mastery
lessons	id, title, level, objective, content, skill_ids
daily_plans	user_id, date, tasks, completion, generated_reason
scenarios	id, category, level, role, objective, difficulty
assessment_reports	user_id, assessment_type, scores, recommendations

progress_snapshots	user_id, date, fluency, grammar, vocab, pronunciation, listening, confidence
settings	user_id, coach_voice, pace, correction_style, explanation_language

11. API Blueprint

```

POST  /api/auth/...
GET   /api/me
PUT   /api/me/profile

POST  /api/assessment/start
POST  /api/assessment/answer
POST  /api/assessment/complete
GET   /api/assessment/latest

POST  /api/session/start
POST  /api/session/turn
POST  /api/session/end
GET   /api/session/{id}
GET   /api/sessions

POST  /api/speech/transcribe
POST  /api/speech/synthesize
POST  /api/speech/analyze

GET   /api/plan/today
POST  /api/plan/generate
POST  /api/plan/{id}/complete

GET   /api/mistakes
GET   /api/vocabulary
POST  /api/review/next
POST  /api/review/{id}/answer

GET   /api/progress
GET   /api/progress/weekly
GET   /api/progress/monthly

GET   /api/scenarios
POST  /api/scenarios/{id}/start
POST  /api/interview/start

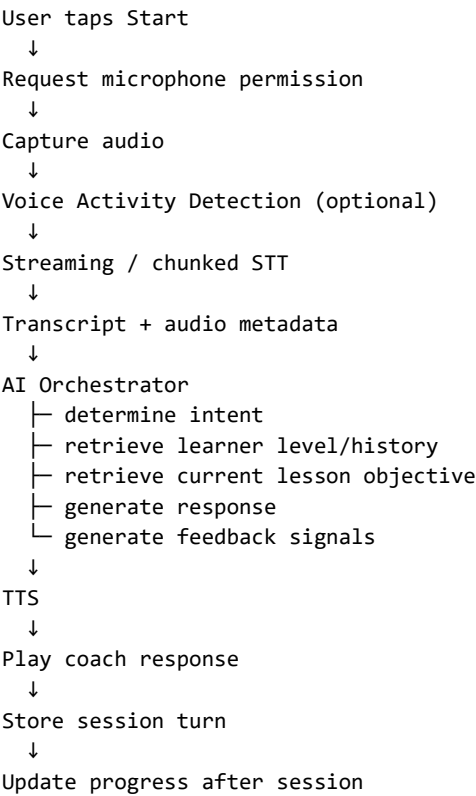
```

12. Main Screens

1. Welcome / onboarding
2. Initial speaking assessment
3. Home / Today's Practice
4. Voice Coach session
5. Guided lesson
6. Free speaking
7. Scenario selection
8. Interview practice
9. Listening practice
10. Pronunciation practice

- 11. Vocabulary & phrases
- 12. My mistakes
- 13. Spaced repetition review
- 14. Progress dashboard
- 15. Weekly report
- 16. Settings / voice / language / privacy

13. Voice Session Technical Workflow



For low latency, use streaming audio/STT/TTS where the selected providers support it. Otherwise implement a reliable turn-based fallback first.

14. Scoring Model

Metric	What to measure	Notes
Fluency	Speech continuity, pauses, hesitation, response completeness	Avoid treating fast speech as automatically better
Grammar	Error types, frequency, severity, corrected usage	Track recurring patterns
Vocabulary	Range, appropriateness, reuse of learned words	Reward useful natural language
Pronunciation	Word/phoneme accuracy, clarity, stress when supported	Use specialist speech assessment where possible
Listening	Comprehension accuracy, dictation accuracy	Adapt difficulty
Confidence	Self-rating + hesitation/response behavior	Label as an estimate, not a medical/psychological measurement

Communication	Clarity, structure, relevance, ability to continue conversation	Primary product metric
---------------	---	------------------------

15. Security, Privacy & Data Handling

- Use HTTPS everywhere.
- Hash passwords if password auth is enabled; prefer OAuth/passwordless where practical.
- Encrypt API keys and provider credentials at rest.
- Use short-lived access tokens and secure refresh-token storage.
- Private audio URLs should be signed/expiring.
- Allow deletion of sessions and recordings.
- Give the user a clear recording consent indicator.
- Do not retain raw audio forever by default; make retention configurable.
- Separate user data by user_id and enforce authorization at every API boundary.
- Keep provider secrets only on the backend.
- Log metadata needed for debugging without storing unnecessary sensitive speech content.
- Because voice recordings are personal data, document the retention and deletion policy before production use.

16. Cost-Control Strategy for Personal Use

- Do not build a multi-tenant SaaS billing system initially.
- Use one personal account with an admin-only deployment if desired.
- Use a provider abstraction so LLM/STT/TTS can be switched later.
- Prefer streaming only where it improves the experience; otherwise use turn-based calls.
- Store transcripts and derived scores; delete raw audio after a configurable period.
- Cache generated lesson content where safe.
- Use small/cheap models for classification, correction and lesson generation; reserve stronger models for difficult tasks.
- Run scheduled analytics asynchronously.
- Add daily/monthly usage counters so API costs cannot silently grow.
- Keep a local-model fallback for non-realtime tasks if hardware permits.

17. Implementation Roadmap

Phase	Build	Definition of Done
0 — Foundation	Repo, architecture, auth, DB, env config, Docker, CI	App boots locally and deploys
1 — Voice Coach MVP	Voice input, STT, LLM, TTS, session history	User can have a reliable spoken coaching session
2 — Assessment	Baseline test + scoring + profile	System creates learner profile
3 — Guided Learning	Daily plan, grammar, vocabulary, translation practice	Daily lesson adapts to weaknesses
4 — Feedback Engine	Mistakes, corrections, pronunciation/clarity, session report	Every session ends with useful feedback
5 — Scenarios	Interviews, workplace, daily-life roleplays	User can practice realistic situations
6 — Listening + Review	Listening, dictation, spaced	Weak skills return automatically

	repetition	
7 — Progress	Dashboard, weekly reports, trends	User can see improvement over time
8 — Polish	Latency, UX, PWA/mobile, observability	Stable personal product
9 — Optional	Avatar, peer practice, multi-user SaaS, billing	Only if later needed

18. Master Prompt / Instructions for a Coding Agent

Paste the following section into Codex/Claude Code/another coding agent as the project brief. The agent should treat this document as the source of truth.

You are building a voice-first AI English Speaking Coach.

PRIMARY OBJECTIVE

Build a personal-use application whose sole purpose is to improve spoken English communication. This is NOT a general AI chat application. Do not create a chat-first UI.

PRODUCT PRINCIPLES

1. Voice first.
2. Speaking practice every session.
3. Structured learning, not random conversation.
4. Personalization from mistakes and performance.
5. Short, useful feedback.
6. Real-world communication over textbook memorization.
7. Tamil/native-language support when the learner is blocked.
8. Every session should contribute to measurable progress.

REQUIRED MODULES

- Onboarding
- Initial speaking assessment
- AI voice coach
- Guided conversation
- Free speaking/extempore
- Real-life scenarios
- Interview practice
- Grammar coaching
- Pronunciation/speech clarity analysis
- Listening practice
- Dictation
- Tamil-to-English speaking practice
- Vocabulary/phrases
- Mistake memory
- Spaced repetition
- Daily personalized plan
- Progress dashboard
- Weekly report
- Lightweight streak/gamification

TECH STACK

Frontend: Next.js + TypeScript + Tailwind + shadcn/ui

Backend: Python + FastAPI

DB: PostgreSQL

Vector: pgvector

Cache/queue: Redis

Storage: S3-compatible object storage

AI: provider abstraction supporting multiple LLM/STT/TTS providers

Deployment: Vercel + Render/Railway/Fly.io initially
Docker for local development

IMPORTANT ARCHITECTURE RULE

Create interfaces for LLM, STT, TTS and pronunciation assessment. Never hard-code provider-specific calls throughout the application.

QUALITY RULES

- Type-safe frontend.
- Pydantic validation in backend.
- Secure secrets.
- Proper auth/authorization.
- Tests for core learning logic.
- Background jobs for heavy audio/report processing.
- Structured logs.
- Clear error states and retry behavior.
- Mobile-first responsive UI.
- No fake pronunciation accuracy claims.
- Never expose provider API keys to frontend.

DEVELOPMENT ORDER

1. Foundation
2. Voice coach
3. Assessment
4. Learning engine
5. Feedback/mistake memory
6. Scenarios/interview
7. Listening/review
8. Progress
9. Polish

DO NOT

- Build a generic chatbot.
- Add unrelated AI features.
- Add billing before the personal version works.
- Over-engineer microservices.
- Add 20 integrations.
- Store raw voice recordings forever.
- Make unsupported claims about pronunciation accuracy.

FIRST DELIVERABLE

Before coding, produce:

- A. final architecture
- B. folder structure
- C. database schema
- D. API contract
- E. environment variables
- F. local setup
- G. implementation task list

Then implement Phase 0 and Phase 1 completely before moving forward.

19. Testing & Acceptance Criteria

- User can complete onboarding without typing long forms.
- Microphone permission failures are handled gracefully.
- User can start a voice session and hear a natural response.

- Conversation does not loop or repeat questions unnecessarily.
- Session transcript is stored correctly.
- End-of-session report contains actionable feedback.
- Recurring mistakes are persisted and influence future lessons.
- Daily plan changes based on previous performance.
- Spaced repetition selects due items correctly.
- Progress charts use stored session data and do not fabricate scores.
- User can delete stored sessions and recordings.
- Provider failures show a retry/fallback message instead of breaking the app.
- Mobile layout works comfortably for one-handed use.

20. Optional Future Enhancements

- AI avatar with lip sync.
- Shadowing mode with model speech.
- Accent-specific coaching.
- Peer-to-peer speaking practice.
- Human mentor integration.
- Android/iOS native app.
- Offline vocabulary review.
- Personal voice coach personalities.
- Advanced pronunciation heatmaps.
- IELTS speaking mode.
- Public speaking mode.
- Business English curriculum.
- Multi-user SaaS with teams and subscriptions.

21. Reference Notes

This specification is an original implementation plan. It is informed by publicly described product capabilities, not by copying proprietary source code, UI assets, prompts or internal systems.

Supernova's public site describes an AI-enabled spoken-English course covering reading, listening, grammar and vocabulary with interactive activities. MySivi's public materials describe AI English coaching focused on real-world communication, pronunciation, grammar, vocabulary, fluency, personalized practice and remembering weaknesses. These references were used only to shape the feature requirements.

- Supernova public product: app.getsupernova.ai
- MySivi public product/content: mysivi.ai and its public blog

22. Gemini API Implementation & Credential Setup

Primary AI provider: Google Gemini API is the default AI provider for V1. OpenAI/Anthropic are not required for the initial build.

Required credential: GEMINI_API_KEY. The developer already has a Gemini API key. Store it only in backend environment/secrets.

Environment variable: GEMINI_API_KEY=your_gemini_api_key_here

Security: Never expose the key in frontend code, browser JavaScript, public repositories, logs, screenshots, or API responses.

Backend flow: Next.js frontend → FastAPI backend → Gemini API. The frontend never calls Gemini with the secret key.

Provider abstraction: Implement an AIProvider interface with Gemini as the first implementation, so another provider can be added later without rewriting the learning engine.

Voice: Prefer a Gemini audio/Live workflow where it provides the needed low-latency capabilities. Otherwise use dedicated STT/TTS services behind interfaces.

Pronunciation: Do not claim precise phoneme/accent scoring from Gemini alone. Add specialist pronunciation assessment later if required.

Cost controls: Use configurable daily/session limits, concise responses, usage logging, and graceful provider-error handling.

Minimum V1 credentials: Gemini API key + PostgreSQL DATABASE_URL. Redis, OAuth, object storage and dedicated pronunciation APIs are optional until needed.

23. Gemini-First Environment Variables

```
# AI
GEMINI_API_KEY=

# DATABASE
DATABASE_URL=

# APPLICATION
APP_ENV=development
APP_URL=http://localhost:3000
BACKEND_URL=http://localhost:8000

# SECURITY
JWT_SECRET=

# Optional later
REDIS_URL=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
S3_ENDPOINT=
S3_ACCESS_KEY=
S3_SECRET_KEY=
S3_BUCKET=
```

PRONUNCIATION_API_KEY=

24. Gemini-First Development Rules for Antigravity

- Treat this document as the source of truth for product scope.
- Build Gemini integration first; do not add OpenAI merely because it appears in older architecture examples.
- Keep AI, STT, TTS and pronunciation behind clean interfaces.
- Implement and test one phase at a time; do not generate the entire application in one pass.
- Inspect existing code before each phase and preserve working functionality.
- Never hard-code secrets or expose GEMINI_API_KEY to the frontend.
- Use structured Gemini prompts and typed JSON responses where practical for assessment, feedback, lesson plans and progress updates.
- Do not turn the product into a generic chatbot; every AI interaction must serve English communication improvement.
- Do not fabricate pronunciation, fluency or progress scores.
- Keep the first deployment simple and avoid unnecessary microservices.

25. Updated First Deliverable for Antigravity

17. Final Gemini-first architecture
18. Folder/monorepo structure
19. PostgreSQL schema and migrations
20. FastAPI API contract
21. Gemini provider service and configuration
22. STT/TTS/pronunciation provider interfaces
23. Environment-variable template
24. Local setup instructions
25. Testing strategy
26. Phased implementation task list
27. Phase 0 Foundation implementation
28. Phase 1 Gemini-powered Voice Coach implementation